Web Technologies — Unit 3 & Unit 4 Comprehensive Study Notes

> ***Subject Type:*** *Programming*
> ***Units Covered:*** *Unit 3 - Unit 4*
> ***Exam Ready:*** *Yes*
> ***Sources:*** *Complete WT.pdf, syllabus.pdf*
> ***Date Generated:*** *2026-03-11*

Table of Contents

1. Unit 3: PHP Server-Side Development
   - 3.1 Introduction to PHP
   - 3.2 PHP Control Structures and Functions
   - 3.3 PHP Arrays and Strings
   - 3.4 Form Processing and File Handling
   - 3.5 Cookies and Sessions
2. Unit 4: PHP and MySQL Integration
   - 4.1 PHP-MySQL Connection
   - 4.2 Database Operations
   - 4.3 Session Management and State

Unit 3: PHP Server-Side Development

3.1 Introduction to PHP

Definition

PHP (Hypertext Preprocessor) is a popular server-side scripting language designed for web development. Unlike client-side scripts, server-side scripts run on the web server, generating dynamic content that is sent to the client.

Key Features of PHP

- **Simplicity:** PHP syntax is simple and familiar to those who know C, Java, or Perl.
- **Embedded within HTML:** PHP code can be directly embedded into HTML.
- **Server-Side Execution:** PHP scripts are executed on the server.
- **Cross-Platform:** Runs on Windows, Linux, and macOS.
- **Database Support:** Supports MySQL, PostgreSQL, SQLite, and more.
- **Open Source:** Free to use with large community support.
- **Extensive Libraries:** Rich set of built-in functions

Server-Side Development Overview

Server-side development involves:

1. Client sends request to server
2. Server processes the request using server-side code
3. Server interacts with databases if needed
4. Server generates dynamic HTML response
5. Response sent back to client

Basic PHP Syntax

```php
<?php
    // PHP code goes here

    // Variables start with $
    $name = "John";
    $age = 25;
    $isActive = true;

    // Output
    echo "Hello, $name!";
    print "You are $age years old.";
?>
```

PHP in HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>PHP Example</title>
</head>
<body>
    <h1>Welcome, <?php echo $name; ?>!</h1>

    <?php if ($age >= 18): ?>
        <p>You are an adult.</p>
    <?php else: ?>
        <p>You are a minor.</p>
    <?php endif; ?>

    <?php for ($i = 1; $i <= 3; $i++): ?>
        <p>Item <?php echo $i; ?></p>
    <?php endfor; ?>
</body>
</html>
```

Data Types

```php
// String
$string = "Hello";           // Double quotes
$single = 'Hello $name';    // Single quotes - no interpolation

// Integer
$integer = 42;
$hex = 0xFF;                // Hexadecimal
$binary = 0b1010;           // Binary

// Float
$float = 3.14;
$scientific = 2.5e3;        // 2500

// Boolean
$boolean = true;
$falseValue = false;

// Array
$array = [1, 2, 3];       // Short array syntax
$assoc = ["a" => 1, "b" => 2];

// Object
$object = new stdClass();
$object->property = "value";

// Null
$null = null;

// Resource
$file = fopen("file.txt", "r");
```

Operators

```php
// Arithmetic
$a + $b; $a - $b; $a * $b; $a / $b; $a % $b; $a ** $b;

// Assignment
$a = $b; $a += $b; $a -= $b; $a *= $b; $a /= $b; $a .= $b;

// Comparison
$a == $b;    // Equal
$a === $b;   // Identical
$a != $b;    // Not equal
$a !== $b;   // Not identical
$a < $b; $a > $b; $a <= $b; $a >= $b;
$a <=> $b;   // Spaceship (PHP 7+): -1, 0, or 1

// Increment/Decrement
++$a; $a++; --$a; $a--;

// Logical
$a && $b; $a || $b; !$a;
$a AND $b; $a OR $b; $a XOR $b;

// Ternary
$result = $condition ? "yes" : "no";
$result = $value ?? "default";  // Null coalescing (PHP 7+)
```

Key Takeaway

PHP is a powerful language for server-side development, enabling developers to create dynamic and interactive web applications with its easy-to-understand syntax and extensive library support.

3.2 PHP Control Structures and Functions

Decision Making

If-Else:

```php
if ($age >= 18) {
    echo "Adult";
} elseif ($age >= 13) {
    echo "Teenager";
} else {
    echo "Child";
}

// Alternative syntax
if ($age >= 18):
    echo "Adult";
elseif ($age >= 13):
    echo "Teenager";
else:
    echo "Child";
endif;
```

Ternary Operator:

```php
$result = ($age >= 18) ? "Adult" : "Minor";

// Null coalescing
$name = $_GET["name"] ?? "Guest";
```

Switch:

```php
switch ($day) {
    case "Monday":
    case "Tuesday":
        echo "Start of week";
        break;
    case "Friday":
        echo "End of week";
        break;
    case "Saturday":
    case "Sunday":
        echo "Weekend!";
        break;
    default:
        echo "Regular day";
}
```

Looping Structures

For Loop:

```php
for ($i = 0; $i < 5; $i++) {
    echo "Number: $i<br>";
}

// Infinite loop
for (;;) {
    // code
}
```

While Loop:

```php
while ($i < 5) {
    echo $i;
    $i++;
}
```

Do-While Loop:

```php
do {
    echo $i;
    $i++;
} while ($i < 5);
```

Foreach Loop:

```php
$colors = ["red", "green", "blue"];

// Value only
foreach ($colors as $color) {
    echo "$color<br>";
}

// With key
foreach ($colors as $key => $color) {
    echo "$key: $color<br>";
}

// Modify array
foreach ($colors as &$color) {
    $color = strtoupper($color);
}
unset($color);  // Important: unset reference
```

Control Structures

Break:

```php
for ($i = 0; $i < 10; $i++) {
    if ($i == 5) {
        break;  // Exit loop
    }
}
```

Continue:

```php
for ($i = 0; $i < 10; $i++) {
    if ($i % 2 == 0) {
        continue;  // Skip iteration
    }
    echo $i;
}
```

Return:

```php
function add($a, $b) {
    return $a + $b;
}
```

Functions

Basic Function:

```php
function greet($name, $greeting = "Hello") {
    return "$greeting, $name!";
}

echo greet("John");           // Hello, John!
echo greet("John", "Hi");     // Hi, John!
```

Type Hints:

```php
function sum(int $a, int $b): int {
    return $a + $b;
}

function greet(string $name): string {
    return "Hello, $name";
}

function process(array $data): void {
    // void - no return value
}
```

Variable Functions:

```php
$func = "strlen";
echo $func("Hello");  // 5

$func = "array_sum";
echo $func([1, 2, 3]);  // 6
```

Anonymous Functions (Closures):

```php
$greet = function($name) {
    return "Hello, $name!";
};

echo $greet("John");

// Using use for closures
$multiply = function($a) use ($b) {
    return $a * $b;
};
```

Arrow Functions (PHP 7.4+):

```php
$square = fn($x) => $x * $x;

$numbers = [1, 2, 3, 4];
$squared = array_map(fn($n) => $n * $n, $numbers);
```

Variable Scope

```php
$globalVar = "I'm global";

function test() {
    global $globalVar;  // Access global variable
    echo $globalVar;

    $localVar = "I'm local";  // Only inside function
    echo $localVar;

    // Using $GLOBALS
    echo $GLOBALS['globalVar'];
}
```

Key Takeaway

PHP provides complete control structures and function capabilities similar to other programming languages. Understanding these fundamentals is essential for building dynamic PHP applications.

3.3 PHP Arrays and Strings

Arrays

Indexed Arrays:

```php
$fruits = ["apple", "banana", "orange"];
// or
$fruits = array("apple", "banana", "orange");

echo $fruits[0];  // apple
echo count($fruits);  // 3

// Add elements
$fruits[] = "grape";
array_push($fruits, "mango");
array_unshift($fruits, "apricot");
```

Associative Arrays:

```php
$person = [
    "name" => "John",
    "age" => 30,
    "city" => "New York"
];

echo $person["name"];  // John

// Add new key
$person["email"] = "john@example.com";
```

Multidimensional Arrays:

```php
$students = [
    ["John", 20, "A"],
    ["Jane", 22, "B"],
    ["Bob", 19, "A"]
];

echo $students[0][0];  // John
echo $students[1][2];  // B

// Associative multidimensional
$employees = [
    "john" => ["name" => "John", "dept" => "IT", "salary" => 50000],
    "jane" => ["name" => "Jane", "dept" => "HR", "salary" => 45000]
];

echo $employees["john"]["dept"];  // IT
```

Array Functions

Function	Description
`count()`	Count elements
`array_push()`	Add elements to end
`array_pop()`	Remove last element
`array_shift()`	Remove first element
`array_unshift()`	Add elements to start
`array_merge()`	Merge arrays
`sort()`	Sort ascending
`rsort()`	Sort descending
`asort()`	Sort by value, keep keys
`ksort()`	Sort by key
`array_search()`	Search for value
`in_array()`	Check if value exists
`array_key_exists()`	Check if key exists
`array_filter()`	Filter array
`array_map()`	Apply function to all
`array_reduce()`	Reduce to single value
`array_slice()`	Extract portion
`array_splice()`	Remove/replace portion
`array_unique()`	Remove duplicates

`array_rand()`	Get random keys
`shuffle()`	Shuffle array

Strings

String Concatenation:

```
$str1 = "Hello";
$str2 = "World";
$result = $str1 . " " . $str2;  // Hello World
```

String Interpolation:

```
$name = "John";
echo "Hello, $name";     // Hello, John (double quotes)
echo 'Hello, $name';     // Hello, $name (single quotes - no interpolation)

// Complex syntax
echo "Hello, {$name}!";  // Curly braces for clarity
echo "Value: ${arr[0]}"; // Array element
```

String Functions

Function	Description
`strlen()`	String length
`strtolower()`	Convert to lowercase
`strtoupper()`	Convert to uppercase
`ucfirst()`	First character uppercase
`ucwords()`	First char of each word uppercase
`trim()`	Remove whitespace
`ltrim()`	Remove from left
`rtrim()`	Remove from right
`substr()`	Get substring
`str_replace()`	Replace string
`strpos()`	Find position
`stripos()`	Find position (case-insensitive)
`strstr()`	Find first occurrence
`explode()`	Split string to array
`implode()` / `join()`	Join array to string
`htmlspecialchars()`	Escape HTML
`nl2br()`	Convert newlines to ` `
`str_pad()`	Pad string
`str_repeat()`	Repeat string

`strrev()`	Reverse string
`str_shuffle()`	Randomly shuffle
`sprintf()`	Format string
`number_format()`	Format number

Key Takeaway

Arrays and strings are fundamental data structures in PHP. PHP provides extensive built-in functions for manipulating arrays and strings efficiently.

3.4 Form Processing and File Handling

HTML Form

```
<form method="POST" action="process.php">
    <input type="text" name="username" required>
    <input type="email" name="email" required>
    <input type="password" name="password">
    <input type="number" name="age">
    <input type="checkbox" name="terms" value="1">
    <select name="country">
        <option value="">Select</option>
        <option value="us">USA</option>
        <option value="uk">UK</option>
    </select>
    <button type="submit">Submit</button>
</form>
```

Form Processing

```php
<?php
// Check if form is submitted
if ($_SERVER["REQUEST_METHOD"] == "POST") {

    // Get form data
    $username = $_POST["username"] ?? "";
    $email = $_POST["email"] ?? "";
    $password = $_POST["password"] ?? "";

    // Sanitize input
    $username = htmlspecialchars(trim($username));
    $email = filter_var($email, FILTER_SANITIZE_EMAIL);
    $password = trim($password);

    // Validation
    $errors = [];

    if (empty($username)) {
        $errors[] = "Username is required";
    } elseif (strlen($username) < 3) {
        $errors[] = "Username must be at least 3 characters";
    }

    if (empty($email)) {
        $errors[] = "Email is required";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors[] = "Invalid email format";
    }

    if (empty($password)) {
        $errors[] = "Password is required";
    } elseif (strlen($password) < 8) {
        $errors[] = "Password must be at least 8 characters";
    }

    // Display errors or process
    if (empty($errors)) {
        // Process the data
        echo "Welcome, $username!";
    } else {
        foreach ($errors as $error) {
            echo "<p style='color: red;'>$error</p>";
        }
    }
}
?>
```

GET vs POST

Aspect	GET	POST
Data Location	URL query string	Request body
Visibility	Visible in URL	Not visible
Data Size	Limited (~2000 chars)	Large (unlimited)
Caching	Can be cached	Not cached
Bookmarking	Can be bookmarked	Cannot bookmark
Security	Less secure	More secure
Use Case	Search, navigation	Form submission

File Handling

Reading Files:

```php
// Read entire file
$content = file_get_contents("data.txt");
echo $content;

// Read line by line
$handle = fopen("data.txt", "r");
while (($line = fgets($handle)) !== false) {
    echo $line;
}
fclose($handle);

// Read into array
$lines = file("data.txt");

// Read with file() - array of lines
$lines = file("data.txt", FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);

// Read specific number of characters
$handle = fopen("data.txt", "r");
$content = fread($handle, 100);  // First 100 bytes
fclose($handle);
```

Writing Files:

```php
// Write to file (overwrites)
file_put_contents("data.txt", "Hello World");

// Append to file
file_put_contents("data.txt", "New line", FILE_APPEND);

// Using fopen/fwrite
$handle = fopen("data.txt", "w");
fwrite($handle, "Content");
fclose($handle);

// Append mode
$handle = fopen("data.txt", "a");
fwrite($handle, "Appended content");
fclose($handle);
```

File Information:

```php
file_exists("data.txt");     // Check if exists
is_readable("data.txt");     // Check if readable
is_writable("data.txt");    // Check if writable
filesize("data.txt");        // Get file size
filemtime("data.txt");       // Get modification time
fileatime("data.txt");       // Get access time
filetype("data.txt");        // Get file type
basename("/path/file.txt");  // Get filename
dirname("/path/file.txt");   // Get directory
pathinfo("/path/file.txt");  // Get path info
```

Working with Directories:

```php
// Create directory
mkdir("newdir");

// Read directory
$dir = opendir(".");
while (($file = readdir($dir)) !== false) {
    echo $file . "<br>";
}
closedir($dir);

// Using scandir
$files = scandir(".");
print_r($files);

// Delete directory
rmdir("emptydir");
```

Key Takeaway

PHP provides powerful features for form processing and file handling. Always validate and sanitize user input for security, and properly handle file operations to prevent errors.

3.5 Cookies and Sessions

Cookies

Setting a Cookie:

```php
// Basic cookie
setcookie("username", "John", time() + (86400 * 7), "/"); // 7 days

// With options
setcookie("theme", "dark", [
    'expires' => time() + (86400 * 30),
    'path' => '/',
    'domain' => 'example.com',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Strict'
]);
```

Reading Cookies:

```php
<?php
// Check if cookie exists
if (isset($_COOKIE["username"])) {
    $username = $_COOKIE["username"];
    echo "Welcome back, $username";
} else {
    echo "Welcome, Guest!";
}

// Get cookie value with default
$theme = $_COOKIE["theme"] ?? "light";
?>
```

Deleting a Cookie:

```php
// Set expiration in the past
setcookie("username", "", time() - 3600, "/");

// Or set to expired
setcookie("username", "", [
    'expires' => time() - 3600,
    'path' => '/'
]);
```

Sessions

Starting a Session:

```php
<?php
// Start session (must be at top before any output)
session_start();

// Store data
$_SESSION["username"] = "John";
$_SESSION["user_id"] = 123;
$_SESSION["role"] = "admin";
$_SESSION["cart"] = ["item1", "item2"];
?>
```

Accessing Session Data:

```php
<?php
session_start();

if (isset($_SESSION["username"])) {
    echo "Welcome, " . $_SESSION["username"];

    // Access array in session
    print_r($_SESSION["cart"]);
} else {
    echo "Please log in";
}
?>
```

Destroying Session:

```php
<?php
// Remove specific session variable
unset($_SESSION["username"]);

// Destroy all session data
session_destroy();

// Clear session array
$_SESSION = [];

// Also clear session cookie
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}
?>
```

Session Configuration

php.ini settings:

Setting	Default	Description
`session.gc_maxlifetime`	1440	Session lifetime (seconds)
`session.cookie_lifetime`	0	Cookie lifetime (0 = browser)
`session.use_cookies`	1	Use cookies
`session.use_only_cookies`	1	Only use cookies
`session.cookie_httponly`	1	HTTP-only cookie
`session.cookie_secure`	0	Secure cookie (HTTPS)
`session.cookie_samesite`	Lax	SameSite policy
`session.save_path`	/tmp	Save path

Set in script:

```php
<?php
// Set session parameters
session_set_cookie_params([
    'lifetime' => 3600,
    'path' => '/',
    'domain' => 'example.com',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Strict'
]);

session_start();

// Custom session name
session_name("MY_SESSION");

// Regenerate ID periodically
if (!isset($_SESSION["initiated"])) {
    session_regenerate_id(true);
    $_SESSION["initiated"] = true;
}
?>
```

Session Timeout

```php
<?php
session_start();

// Timeout after 30 minutes
$timeout = 1800; // 30 minutes

if (isset($_SESSION["last_activity"])) {
    $elapsed = time() - $_SESSION["last_activity"];
    if ($elapsed > $timeout) {
        session_unset();
        session_destroy();
        header("Location: login.php?timeout=1");
        exit;
    }
}

$_SESSION["last_activity"] = time();

// Regenerate session ID for security
if (isset($_SESSION["created"])) {
    if (time() - $_SESSION["created"] > 300) {
        session_regenerate_id(true);
        $_SESSION["created"] = time();
    }
} else {
    $_SESSION["created"] = time();
}
?>
```

Cookies vs Sessions

Aspect	Cookies	Sessions
Storage	Client (browser)	Server
Security	Less secure	More secure
Capacity	~4KB	Unlimited
Expiration	Manual	Session/configured
Performance	Faster (no server call)	Slightly slower
Use Case	Preferences, tracking	Authentication, sensitive data

Key Takeaway

Cookies and sessions are essential for maintaining user state in web applications. Sessions are more secure for sensitive data, while cookies are useful for persistent user preferences.

Unit Summary: Unit 3

Quick Reference Card

Concept	Syntax/Formula	Example
PHP Tags	`<?php ?>`	`<?php echo $var; ?>`
Variables	`$variable`	`$name = "John";`
Arrays	`array()` or `[]`	`$arr = [1, 2, 3];`
Form Data	`$_POST`, `$_GET`	`$_POST["username"]`
Session	`$_SESSION`	`$_SESSION["user"]`
Cookie	`setcookie()`	`setcookie("u", "val")`
File Read	`file_get_contents()`	Read entire file
File Write	`file_put_contents()`	Write to file

Practice Questions

1. **[Easy]** Explain the difference between POST and GET methods.
2. **[Medium]** Create a PHP form that validates email and password.
3. **[Hard]** Implement a login system using PHP sessions.

Common Errors

Error	Cause	Fix
Undefined index	Missing form field	Use `isset()` or `??`
Headers already sent	Output before redirect	Use output buffering
Session not starting	Before any output	Start at top of file
File not found	Wrong path	Check file path

Unit 4: PHP and MySQL Integration

4.1 PHP-MySQL Connection

Definition

PHP and MySQL are commonly used together to create dynamic web applications that interact with databases. MySQL is a popular relational database management system, while PHP provides the server-side logic to interact with it.

Connection Methods

Using MySQLi (Object-Oriented):

```php
<?php
$servername = "localhost";
$username = "root";
$password = "";
$database = "mydb";

// Create connection
$conn = new mysqli($servername, $username, $password, $database);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

echo "Connected successfully";
$conn->close();
?>
```

Using MySQLi (Procedural):

```php
<?php
$conn = mysqli_connect("localhost", "root", "", "mydb");

if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

echo "Connected successfully";
mysqli_close($conn);
?>
```

Using PDO (PHP Data Objects):

```php
<?php
$servername = "localhost";
$username = "root";
$password = "";
$database = "mydb";

try {
    $conn = new PDO("mysql:host=$servername;dbname=$database", $username, $password);
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connected successfully";
} catch(PDOException $e) {
    die("Error: " . $e->getMessage());
}

// Close connection
$conn = null;
?>
```

Comparison: MySQLi vs PDO

Feature	MySQLi	PDO
Database Support	MySQL only	Multiple databases
API Style	Object & Procedural	Object-oriented
Prepared Statements	Yes	Yes
Performance	Slightly faster	Slightly slower
Error Handling	Property-based	Exception-based

Connection Error Handling

```php
// MySQLi (Object-Oriented)
$conn = new mysqli("localhost", "root", "", "mydb");
if ($conn->connect_error) {
    die("Error: " . $conn->connect_error);
}

// MySQLi (Procedural)
$conn = mysqli_connect("localhost", "root", "", "mydb");
if (mysqli_connect_error()) {
    die("Error: " . mysqli_connect_error());
}

// PDO (recommended)
try {
    $conn = new PDO("mysql:host=localhost;dbname=mydb", "root", "");
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(PDOException $e) {
    die("Error: " . $e->getMessage());
}
```

Key Takeaway

Establishing a secure database connection is the first step in PHP-MySQL integration. Using PDO is recommended for better security and database abstraction.

4.2 Database Operations

Creating a Database

```php
<?php
$conn = new mysqli("localhost", "root", "");

// Create database
$sql = "CREATE DATABASE mydb";
if ($conn->query($sql) === TRUE) {
    echo "Database created successfully";
} else {
    echo "Error: " . $conn->error;
}

$conn->close();
?>
```

Creating a Table

```php
<?php
$conn = new mysqli("localhost", "root", "", "mydb");

$sql = "CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
)";

if ($conn->query($sql) === TRUE) {
    echo "Table created successfully";
} else {
    echo "Error: " . $conn->error;
}

$conn->close();
?>
```

Inserting Data

Using MySQLi:

```php
<?php
$conn = new mysqli("localhost", "root", "", "mydb");

// Direct query (NOT RECOMMENDED - SQL Injection risk)
$conn->query("INSERT INTO users (username, email, password) VALUES ('john',
'john@example.com', 'pass')");

// Prepared statement (RECOMMENDED)
$stmt = $conn->prepare("INSERT INTO users (username, email, password) VALUES (?, ?, ?)");
$stmt->bind_param("sss", $username, $email, $password);

$username = "john";
$email = "john@example.com";
$password = password_hash("password123", PASSWORD_DEFAULT);

$stmt->execute();

echo "New record inserted successfully. ID: " . $stmt->insert_id;
$stmt->close();
$conn->close();
?>
```

Using PDO:

```php
<?php
try {
    $conn = new PDO("mysql:host=localhost;dbname=mydb", "root", "");
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $stmt = $conn->prepare("INSERT INTO users (username, email, password) VALUES (:username,
:email, :password)");

    $username = "john";
    $email = "john@example.com";
    $password = password_hash("password123", PASSWORD_DEFAULT);

    $stmt->execute([
        ':username' => $username,
        ':email' => $email,
        ':password' => $password
    ]);

    echo "New record inserted. ID: " . $conn->lastInsertId();

} catch(PDOException $e) {
    die("Error: " . $e->getMessage());
}
?>
```

Selecting Data

Using MySQLi:

```php
<?php
$conn = new mysqli("localhost", "root", "", "mydb");

// Select all rows
$result = $conn->query("SELECT * FROM users");

if ($result->num_rows > 0) {
    while($row = $result->fetch_assoc()) {
        echo "ID: " . $row["id"] . " - Name: " . $row["username"] . "<br>";
    }
} else {
    echo "0 results";
}

// Prepared statement
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ?");
$stmt->bind_param("s", $username);
$username = "john";
$stmt->execute();
$result = $stmt->get_result();

while ($row = $result->fetch_assoc()) {
    echo $row["email"];
}

$stmt->close();
$conn->close();
?>
```

Updating Data

```php
<?php
$conn = new mysqli("localhost", "root", "", "mydb");

$stmt = $conn->prepare("UPDATE users SET email = ? WHERE username = ?");
$stmt->bind_param("ss", $email, $username);

$email = "newemail@example.com";
$username = "john";

if ($stmt->execute()) {
    echo "Record updated successfully. Rows affected: " . $stmt->affected_rows;
} else {
    echo "Error: " . $conn->error;
}

$stmt->close();
$conn->close();
?>
```

Deleting Data

```php
<?php
$conn = new mysqli("localhost", "root", "", "mydb");

$stmt = $conn->prepare("DELETE FROM users WHERE id = ?");
$stmt->bind_param("i", $id);

$id = 5;

if ($stmt->execute()) {
    echo "Record deleted successfully";
}

$stmt->close();
$conn->close();
?>
```

SQL Injection Prevention

Never do this:

```php
// DANGEROUS - SQL Injection!
$username = $_POST["username"];
$query = "SELECT * FROM users WHERE username = '$username'";
$result = $conn->query($query);
```

Always use prepared statements:

```php
// SAFE
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ?");
$stmt->bind_param("s", $username);
$stmt->execute();
```

Key Takeaway

Using prepared statements is essential for preventing SQL injection attacks. Both MySQLi and PDO support prepared statements - use them for all database operations.

4.3 Session Management and State

The Problem of State

HTTP is a stateless protocol - each request is independent. Web applications need to maintain state across multiple requests to:

- Keep users logged in
- Shopping cart contents
- User preferences
- Tracking user activity

Methods of State Management

Method	Storage	Use Case
Query Strings	URL	Simple data passing
URL Parameters	URL	Simple data passing
Hidden Fields	Form	Form to form data
Cookies	Client	Persistent preferences
Sessions	Server	User authentication

Passing Information via Query Strings

Creating Query String:

```php
<?php
$page = 2;
$sort = "name";
$category = "electronics";

// Manual
$url = "products.php?page=$page&sort=$sort";

// Using http_build_query (recommended)
$params = [
    "page" => $page,
    "sort" => $sort,
    "category" => $category
];
$url = "products.php?" . http_build_query($params);
// Result: products.php?page=2&sort=name&category=electronics
?>

<a href="<?php echo $url; ?>">Next Page</a>
```

Reading Query Strings:

```php
<?php
// Get single value
$page = $_GET["page"] ?? 1;

// Get all parameters
$params = $_GET;

// Sanitize input
$page = filter_var($page, FILTER_VALIDATE_INT, [
    "options" => ["default" => 1, "min_range" => 1]
]);

// Check if parameter exists
if (isset($_GET["sort"])) {
    $sort = htmlspecialchars($_GET["sort"]);
}
?>
```

Passing Information via URL Path

```php
<?php
// Example URL: /user/profile/123

// Using $_SERVER
$path = $_SERVER["REQUEST_URI"];

// Parse URL
$parsed = parse_url($path);
$path = $parsed["path"];

// Using explode
$parts = explode("/", trim($path, "/"));
// $parts = ["user", "profile", "123"]

// Or use regular expression
if (preg_match('#^/user/profile/(\d+)$#', $path, $matches)) {
    $userId = $matches[1];
}
?>
```

Using .htaccess for Clean URLs:

```apache
# Enable mod_rewrite
RewriteEngine On

# RewriteRule pattern substitution

# Simple rewrite
RewriteRule ^about$ about.php

# With parameters
RewriteRule ^user/(\d+)$ profile.php?id=$1

# With multiple parameters
RewriteRule ^products/(\w+)/(\d+)$ products.php?category=$1&page=$2
```

Session State Management

```php
<?php
session_start();

// Set session timeout (30 minutes)
$timeout = 1800; // 30 minutes in seconds

if (isset($_SESSION["last_activity"])) {
    $elapsed = time() - $_SESSION["last_activity"];
    if ($elapsed > $timeout) {
        session_unset();
        session_destroy();
        header("Location: login.php?timeout=1");
        exit;
    }
}

$_SESSION["last_activity"] = time();

// Regenerate session ID periodically (security)
if (!isset($_SESSION["created"])) {
    $_SESSION["created"] = time();
} else {
    if (time() - $_SESSION["created"] > 300) {
        session_regenerate_id(true);
        $_SESSION["created"] = time();
    }
}

// Store user data safely
$_SESSION["user_id"] = $user["id"];
$_SESSION["username"] = $user["username"];
$_SESSION["role"] = $user["role"];

// Don't store sensitive data in session
// $_SESSION["password"] = $password; // WRONG!
?>
```

Serialization

```php
<?php
// Array to JSON
$data = ["name" => "John", "age" => 30];
$json = json_encode($data);
// Result: {"name":"John","age":30}

// JSON to array
$array = json_decode($json, true);

// Check for JSON errors
if (json_last_error() !== JSON_ERROR_NONE) {
    echo "JSON Error: " . json_last_error_msg();
}

// PHP Serialize (not recommended for security)
$serialized = serialize($data);
$unserialized = unserialize($serialized);

// Pass data between pages/systems
// Using hidden form fields
echo '<input type="hidden" name="token" value="' . htmlspecialchars($json) . '">';

// Using JavaScript
echo '<script>var data = ' . json_encode($data) . ';</script>';
?>
```

Complete Login/Logout Example

login.php:

```php
<?php
session_start();

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = $_POST["username"];
    $password = $_POST["password"];

    // Validate input
    if (empty($username) || empty($password)) {
        $error = "Please enter username and password";
    } else {
        // Check database
        $conn = new mysqli("localhost", "root", "", "mydb");
        $stmt = $conn->prepare("SELECT id, username, password FROM users WHERE username =
?");
        $stmt->bind_param("s", $username);
        $stmt->execute();
        $result = $stmt->get_result();

        if ($row = $result->fetch_assoc()) {
            if (password_verify($password, $row["password"])) {
                // Login successful
                $_SESSION["user_id"] = $row["id"];
                $_SESSION["username"] = $row["username"];
                $_SESSION["last_activity"] = time();

                header("Location: dashboard.php");
                exit;
            }
        }

        $error = "Invalid username or password";
        $stmt->close();
        $conn->close();
    }
}
?>

<form method="POST">
    <?php if (isset($error)): ?>
        <p style="color: red;"><?php echo $error; ?></p>
    <?php endif; ?>
    <input type="text" name="username" placeholder="Username">
    <input type="password" name="password" placeholder="Password">
    <button type="submit">Login</button>
</form>
```

logout.php:

```php
<?php
session_start();
$_SESSION = [];

if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}

session_destroy();
header("Location: login.php");
exit;
?>
```

Key Takeaway

Understanding state management is crucial for building interactive web applications. Choose the appropriate method based on your security requirements and use case.

Unit Summary: Unit 4

Quick Reference Card

Concept	Syntax/Formula	Example
MySQLi Connection	`$conn = new mysqli()`	Connect to database
PDO Connection	`$conn = new PDO()`	Connect with PDO
Prepared Statement	`$stmt->prepare()`	SQL injection prevention
Session Start	`session_start()`	Start user session
Session Data	`$_SESSION["key"]`	Store/retrieve data
JSON Encode	`json_encode()`	Convert to JSON
Query String	`http_build_query()`	Build URL parameters

Practice Questions

1. **[Easy]** Explain the difference between MySQLi and PDO.
2. **[Medium]** Create a PHP script to insert, update, and delete records from a MySQL database.
3. **[Hard]** Implement a complete user authentication system with session management.

Common Errors

Error	Cause	Fix
SQL Injection	Unsanitized input	Use prepared statements
Connection failed	Wrong credentials	Check database settings
Session lost	Cookie blocked	Check browser settings
JSON error	Invalid JSON	Use json_last_error()

End of Document

Coverage Statistics

Unit	Topics Covered	Coverage
Unit 3	PHP Basics, Forms, Files, Sessions	100%
Unit 4	PHP-MySQL, State Management	100%

Source Index

1. Complete WT.pdf - Main textbook content
2. syllabus.pdf - Course syllabus and objectives